

Python Klausur-Formelsammlung

Spickzettel & Syntaxreferenz | Teil 1 bis Teil 3 & Passives Wissen

Teil 1: Grundlagen, Datentypen & Ablaufsteuerung

■ Programmaufbau, PEP8-Standards & Docstrings

- **PEP 8 Standards:** Offizieller Style Guide für Python-Code.
 - **Einrückung:** Konsequenz 4 Leerzeichen pro Ebene. Keine Tabs.
 - **Zeilenlänge:** Maximal 79 Zeichen für Code, 72 für Docstrings/Kommentare.
 - **Namenskonventionen:**
 - snake_case: Variablen, Funktionen, Methoden, Module (z.B. berechne_wert, data_list).
 - PascalCase: Klassen (z.B. KundenManager, AbstractVehicle).
 - UPPER_CASE: Konstanten (z.B. MAX_INTEGERS, PI = 3.1415).
 - **Docstrings (PEP 257):** Deklaration von Dokumentation direkt unterhalb von Definitionen (def, class) mittels dreifacher Anführungszeichen.
 - **Einzeilig:** `'''Berechnet die Fakultät einer Zahl.'''`
 - **Mehrzeilig:** Erste Zeile als kurze Zusammenfassung, gefolgt von einer Leerzeile und detaillierter Spezifikation von Parametern (Args:) und Rückgabewerten (Returns:).

```
def division(a: float, b: float) -> float:
    '''Dividiert zwei Gleitkommazahlen miteinander.

    Args:
        a (float): Der Dividend.
        b (float): Der Divisor (darf nicht 0 sein).

    Returns:
        float: Das Ergebnis der Division.
    '''
    return a / b
```

■ Python-Datentypen & Eigenschaften

Python-Objekte zeichnen sich primär durch ihre Identität (id()), ihren Typ (type()) und ihren Wert aus. Zwei fundamentale Eigenschaften von Typen sind:

Mutable (Veränderbar): Das Objekt kann nach seiner Erstellung in-place modifiziert werden. Änderungen betreffen dieselbe Speicheradresse.

Iterable (Iterierbar): Das Objekt kann in einer Schleife durchlaufen werden (implementiert iter oder getitem).

Datentyp	Mutable	Iterable	Syntax / Code-Beispiel
int	Nein	Nein	x = -5, y = 42
float	Nein	Nein	pi = 3.1415, e_notation = 2e-3
bool	Nein	Nein	is_active = True, has_error = False
NoneType	Nein	Nein	result = None (Repräsentiert Leere/Nullwert)
list	Ja	Ja	items = [1, "Text", 3.4, [1, 2]]
tuple	Nein	Ja	point = (10, 20), single = (5,)
str	Nein	Ja	text = "Python", multiline = """Line1"""
range	Nein	Ja	r = range(0, 10, 2) (Start, Stop exkl., Step)
dict	Ja	Ja (Keys)	mapping = {"id": 1, "status": True}
set	Ja	Ja	unique_elements = {1, 2, 3, 3} -> {1, 2, 3}

Wichtiges Klausurwissen zu Datentypen:

Dictionaries & Sets: Schlüssel (keys) bzw. Set-Elemente müssen zwingend hashable (unveränderbar, d.h. immutable) sein. Eine Liste kann niemals Schlüssel eines Dictionaries sein!

Tupel: Sind zwar immutable, können aber mutable Elemente enthalten (z.B. eine Liste innerhalb eines Tupels). Die Liste selbst bleibt veränderbar.

■ Geläufige Built-in-Funktionen

`enumerate(iterable, start=0)`: Liefert beim Iterieren Paare bestehend aus Index und Wert.

`zip(*iterables)`: Aggregiert Elemente aus mehreren Iterables parallel zu Tupeln. Bricht ab, sobald das kürzeste Iterable vollständig durchlaufen ist.

`sum(iterable, start=0)`: Summiert alle numerischen Elemente eines Iterables plus den start-Wert.

`sorted(iterable, key=None, reverse=False)`: Gibt eine neue, sortierte Liste zurück. Das Original bleibt unverändert (im Gegensatz zu `list.sort()`).

```
# Anwendung der built-ins
namen = ["Anna", "Bob", "Chris"]
alter = [22, 25]

# zip & enumerate kombiniert
for i, (n, a) in enumerate(zip(namen, alter)):
    print(f"{i}: {n} ist {a} Jahre alt.") # Chris wird ignoriert, da alter zu kurz ist

zahlen = [5, 2, 9, 1]
sortiert = sorted(zahlen, reverse=True) # [9, 5, 2, 1]
total = sum(zahlen, start=10)          # 10 + 5 + 2 + 9 + 1 = 27
```

■ Geläufige String- und Listenmethoden

String-Methoden (Erzeugen immer ein neues Objekt, da Strings immutable sind)

`s.strip()`: Entfernt Whitespaces am Anfang und Ende des Strings.

`s.split(sep=None)`: Zerlegt den String bei `sep` in eine Liste von Teilstrings.

`sep.join(iterable)`: Verbindet die Strings aus einem Iterable mit dem Trennzeichen `sep`.

`s.replace(old, new)`: Ersetzt alle Vorkommen von `old` durch `new`.

`s.lower()` / `s.upper()`: Transformiert alle Zeichen in Klein- bzw. Großbuchstaben.

`s.startswith(prefix)` / `s.endswith(suffix)`: Liefert True oder False.

Listen-Methoden (Modifizieren die Liste meist in-place)

`l.append(x)`: Fügt das Element `x` am Ende der Liste hinzu.

`l.extend(iterable)`: Hängt alle Elemente des Iterables an die Liste an.

`l.insert(index, x)`: Fügt das Element `x` am spezifizierten index ein.

`l.pop(index=-1)`: Entfernt das Element am index (Standard: letztes) und gibt es zurück.

`l.remove(x)`: Entfernt das erste Vorkommen des Wertes `x` (wirft `ValueError`, falls nicht existent).

`l.clear()`: Entfernt alle Elemente aus der Liste.

■ Konzepte der Typisierung

Dynamische Typisierung: Der Typ einer Variablen wird nicht deklariert, sondern zur Laufzeit anhand des zugewiesenen Objekts bestimmt. Variablen sind lediglich Zeiger auf Objekte.

Duck Typing: "Wenn es wie eine Ente geht und wie eine Ente quakt, dann ist es eine Ente." Der explizite Typ eines Objekts ist irrelevant; wichtig ist ausschließlich, ob das Objekt die benötigten Methoden oder Attribute aufweist.

EAFP (Easier to Ask for Forgiveness than Permission): Typisch pythonischer Programmierstil. Man führt Operationen direkt aus und fängt potenzielle Fehler mittels `try/except` ab, statt vorher zeitaufwendig Bedingungen abzufragen.

LBYL (Look Before You Leap): Das Gegenteil von EAFP. Vor der Operation wird explizit geprüft (z.B. via `if os.path.exists(f)`), was in einer Multithreading-Umgebung zu Race Conditions (TOCTOU) führen kann.

Type Hinting: Optionale statische Typannotationen zur Verbesserung der Lesbarkeit und Validierung durch Linters (z.B. mypy). Hat zur Laufzeit keine Auswirkungen auf die Performance oder Typsicherheit.

```
# Duck Typing & Type Hinting Beispiel
def aktiviere_geraet(geraet: any) -> None:
    # Es wird nicht geprüft, ob geraet vom Typ 'Maschine' ist. Hauptsache es hat 'starten()'
    geraet.starten()
```

■ Ablaufsteuerung & Parameter

Bedingte Verzweigungen & Schleifen

if / elif / else: Standardmäßige Verzweigung.

while bedingung:: Schleife läuft, solange die Bedingung wahr ist.

for element in iterable:: Durchläuft ein iterierbares Objekt.

break: Bricht die innerste Schleife sofort ab.

continue: Springt sofort in den nächsten Schleifenlauf.

Match-Case (Structural Pattern Matching, Python 3.10+)

Erlaubt elegantes Pattern Matching, weit über klassische Switch-Statements hinaus.

```
def verarbeite_kommando(kommando):
    match kommando:
        case "start":
            return "System startet"
        case "stop" | "halt": # OR-Verknüpfung
            return "System stoppt"
        case ["move", direction]: # Matching von Sequenzen & Entpacken
            return f"Bewege in Richtung {direction}"
        case _: # Wildcard (default)
            return "Unbekanntes Kommando"
```

Default- und Keyword-Parameter

Default-Parameter: Erlauben das Weglassen von Argumenten beim Aufruf.

Keyword-Parameter: Aufruf erfolgt explizit über den Namen des Parameters, wodurch die Reihenfolge irrelevant wird.

Achtung bei mutablen Default-Parametern (Klausurfalle!):

Default-Argumente werden nur einmalig zum Zeitpunkt der Funktionsdefinition ausgewertet. Wird ein mutables Objekt (wie eine Liste) als Default-Wert übergeben, teilen sich alle späteren Funktionsaufrufe dieselbe Instanz!

```
# ANTI-PATTERN
def append_to(element, target=[]):
    target.append(element)
    return target

print(append_to(1)) # [1]
print(append_to(2)) # [1, 2] -> Überraschung!

# CLEAN-PATTERN
def append_to_clean(element, target=None):
    if target is None:
        target = []
    target.append(element)
    return target
```

■ Algorithmen: Rekursion

Eine Funktion ist rekursiv, wenn sie sich selbst im eigenen Funktionskörper aufruft. Jede Rekursion benötigt zwingend zwei Komponenten:

Basisfall (Base Case): Die Abbruchbedingung, die ohne weiteren rekursiven Aufruf ein Ergebnis liefert. Verhindert Endlosschleifen.

Rekursionsschritt (Recursive Step): Der Selbstaufruf mit einem modifizierten, verkleinerten Problemraum, der sich dem Basisfall annähert.

```
def fakultaet(n: int) -> int:
    if n <= 1: # Basisfall
        return 1
    return n * fakultaet(n - 1) # Rekursionsschritt
```

Gefahr: Jeder rekursive Aufruf belegt Speicherplatz auf dem Call-Stack. Bei zu tiefer Rekursion droht ein RecursionError (Standard-Limit liegt meist bei 1000 Aufrufen).

Teil 2: Fortgeschrittene Konzepte, Fehlerbehandlung & I/O

■ Indizierung, Slicing & List Comprehensions

Slicing-Syntax: `sequence[start:stop:step]`

start: Inklusiver Startindex (Standard: 0).

stop: Exklusiver Endindex (Standard: Länge der Sequenz).

step: Schrittweite (Standard: 1). Negative Schrittweiten durchlaufen rückwärts.

```
a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(a[2:5])    # [2, 3, 4]      (Elemente von Index 2 bis 4)
print(a[:3])     # [0, 1, 2]     (Ersten drei Elemente)
print(a[8:])     # [8, 9]        (Ab Index 8 bis zum Ende)
print(a[1:8:2])  # [1, 3, 5, 7]  (Jedes zweite Element von 1 bis 7)
print(a[::-1])   # [9, 8, 7, ...] (Invertiert die gesamte Liste)
```

List Comprehensions

Eine elegante, performante Methode zur Generierung von Listen aus bestehenden Iterables. Syntax: `[expression for item in iterable if condition]`

```
# Klassisch
quadrates = []
for x in range(10):
    if x % 2 == 0:
        quadrates.append(x**2)

# Per List Comprehension
quadrates_lc = [x**2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]
```

== F-Strings und Formatierung

Ermöglichen die direkte Einbettung von Ausdrücken in String-Literale mit dem Präfix `f`. Zur Formatierung von Zahlen wird ein Doppelpunkt `:` angehängt. Python

```
wert = 12.34567
print(f"Standard: {wert}")      # "Standard: 12.34567"
print(f"Gerundet: {wert:.2f}")  # "Gerundet: 12.35" (Gleitkomma mit 2 Nachkommastellen)
print(f"Padding: {wert:08.2f}") # "Padding: 00012.35" (Gesamtlänge 8, aufgefüllt mit Nullen)

zahl = 42
print(f"Ausrichtung: {zahl:<5}") # "42   " (Linksbündig auf Breite 5)
print(f"Ausrichtung: {zahl:>5}") # "    42" (Rechtsbündig auf Breite 5)
```

■ Funktionen als Objekte (First-Class Functions)

In Python sind Funktionen "First-Class Citizens". Das bedeutet, sie sind vollwertige Objekte und können:

An Variablen zugewiesen werden.

Als Argumente an andere Funktionen übergeben werden.

Als Rückgabewerte aus Funktionen geliefert werden.

```
def quadrat(x): return x * x
def kubik(x): return x * x * x

def wende_an(func, wert): # Funktion als Argument
    return func(wert)
```

```
operationen = [quadrat, kubik] # Funktionen in einer Datenstruktur
print(wende_an(operationen[0], 4)) # Ruft quadrat(4) auf -> 16
```

■ Exceptions & Fehlerbehandlung (try-except-else-finally)

Fehler zur Laufzeit (Ausnahmen) unterbrechen den normalen Kontrollfluss. Sie können gezielt abgefangen werden, um einen Programmabsturz zu verhindern.

Wichtige Built-in Exceptions:

`TypeError`: Operation auf ein Objekt mit unpassendem Datentyp (z.B. `len(5)`).

`ValueError`: Funktion erhält Argument vom richtigen Typ, aber mit ungültigem Wert (z.B. `int("abc")`).

`KeyError`: Zugriff auf einen nicht existierenden Schlüssel im Dictionary.

`IndexError`: Zugriff auf einen Index außerhalb der Grenzen einer Sequenz.

Die vollständige try-Blockstruktur:

```
try:
    # Risiko-Code: Hier könnte eine Exception geworfen werden
    datei = open("daten.txt", "r")
    wert = int(datei.readline())
except FileNotFoundError:
    print("Datei existiert nicht!")
except ValueError as e:
    print(f"Konvertierungsfehler: {e}")
except Exception as e:
    print(f"Generischer Fehlerfänger für alle anderen Exceptions: {e}")
else:
    # Optional: Wird NUR ausgeführt, wenn im try-Block KEINE Exception auftrat
    print(f"Erfolgreich gelesen. Wert ist {wert}")
finally:
    # Optional: Wird IMMER ausgeführt, egal ob ein Fehler auftrat oder nicht
    # Ideal für Aufräumarbeiten (Schließen von Sockets, Dateien, DB-Verbindungen)
    print("Aufräumarbeiten werden durchgeführt.")
```

■ Test-Driven Development (TDD) mit pytest

Der TDD-Entwicklungszyklus folgt dem Prinzip Red - Green - Refactor:

Red: Schreiben eines Unittests für eine noch nicht existierende Funktionalität. Der Test schlägt fehl.

Green: Schreiben des minimal notwendigen Codes, damit der Test erfolgreich verläuft.

Refactor: Bereinigen und Optimieren des Codes unter der Garantie, dass alle Tests weiterhin bestehen.

Pytest-Syntax & Assertions:

Pytest wertet einfache Python-Standard-assert-Statements aus. Schlägt ein Assert fehl, wird ein `AssertionError` geworfen.

```
# Code-Datei: rechner.py
def addiere(a, b):
    return a + b

# Test-Datei: test_rechner.py
import pytest
from rechner import addiere

def test_addiere_positiv():
    assert addiere(2, 3) == 5 # Erwartet True

def test_addiere_fehler():
    # Überprüfen, ob eine spezifische Exception geworfen wird
    with pytest.raises(TypeError):
        addiere("2", 3)
```

■ Module und der Main-Switch

Jede .py-Datei stellt ein Modul dar. Beim Import eines Moduls wird dessen Code vollständig von oben nach unten ausgeführt.

Main-Switch: `if __name__ == "__main__":`

Wird eine Datei direkt ausgeführt, setzt Python die interne Variable `__name__` auf den String `"__main__"`.

Wird die Datei hingegen als Modul importiert, entspricht `__name__` dem eigentlichen Dateinamen. Der Codeblock im Main-Switch wird dann nicht ausgeführt. Dies verhindert unerwünschte Nebeneffekte beim Importieren.

```
# modul.py
def helfer():
    print("Helferfunktion aufgerufen")

if __name__ == "__main__":
    # Dieser Code läuft NUR, wenn modul.py direkt gestartet wird
    print("Modul wird als Hauptprogramm getestet")
    helfer()
```

■ Datei-Handling & Kontextmanager

Das manuelle Öffnen und Schließen von Ressourcen birgt das Risiko von Ressourcen-Leaks, falls das `close()` aufgrund eines Fehlers übersprungen wird.

Kontextmanager (with-Statement): Garantiert das automatische und sichere Schließen der Ressource beim Verlassen des Blocks, selbst wenn im Block Laufzeitfehler auftreten. Ein Kontextmanager implementiert intern die magischen Methoden `__enter__()` und `__exit__()`.

```
# Sicheres Lesen einer Textdatei
with open("beispiel.txt", "r", encoding="utf-8") as datei:
    inhalt = datei.read()
# Nach dem Block ist die Datei automatisch geschlossen.

# CSV-Dateien lesen und schreiben
import csv

# Schreiben einer CSV-Datei
with open("daten.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f, delimiter=";")
    writer.writerow(["ID", "Name", "Rolle"])
    writer.writerows([[1, "Alice", "Admin"], [2, "Bob", "User"]])

# Lesen einer CSV-Datei
with open("daten.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f, delimiter=";")
    for zeile in reader:
        print(zeile) # Liefert Listen von Strings, z.B. ["1", "Alice", "Admin"]
```

Teil 3: Objektorientierte Programmierung (OOP)

■ Konzepte, Klassen, Attribute & Methoden

Die OOP strukturiert Programme durch die Kapselung von Daten (Attributen) und zugehörigen Verhaltensweisen (Methoden) in logischen Einheiten (Klassen).

Klasse: Der abstrakte Bauplan bzw. die Schablone (z.B. Klasse Auto).

Objekt / Instanz: Das konkrete, im Speicher existierende Abbild des Bauplans (z.B. `mein_polo = Auto()`).

self: Ein Zeiger auf die konkrete Instanz des Objekts. Muss als erster Parameter bei jeder Instanzmethode definiert werden.

`__init__`: Die Initialisierungsmethode (Konstruktor). Sie wird automatisch aufgerufen, sobald ein neues Objekt instanziiert wird, um Attribute zu setzen.

```
class Student:
    def __init__(self, name: str, matrikelnummer: int):
        self.name = name # Instanzattribut
        self.matrikelnummer = matrikelnummer # Instanzattribut

    def zeige_info(self): # Instanzmethode
        return f"Student: {self.name} ({self.matrikelnummer})"
```

■ Kapselung (Sichtbarkeiten)

Kapselung schützt den internen Zustand eines Objekts vor unkontrollierter Modifikation von außen. Python kennt keine echten Zugriffsrestriktionen (private via Compiler), steuert dies aber über Namenskonventionen:

Public (name): Von überall uneingeschränkt les- und schreibbar.

Protected (`_name`): Konvention! Signalisiert dem Entwickler, dass auf dieses Attribut nur innerhalb der eigenen Klasse sowie deren Subklassen zugegriffen werden sollte. Technisch verhält es sich wie ein Public-Attribut.

Private (`__name`): Aktiviert das sogenannte Name Mangling (Namensmischung). Python benennt das Attribut intern implizit um in `_Klassenname__name`. Ein direkter Zugriff von außen via `objekt.__name` provoziert einen `AttributeError`.

```
class Tresor:
    def __init__(self, code):
        self.besitzer = "Allgemein" # Public
        self._typ = "Wandtresor"    # Protected
        self.__code = code          # Private (Name Mangling greift)

t = Tresor(1234)
print(t.besitzer)    # Funktioniert
print(t._typ)        # Funktioniert technisch, verletzt aber die Konvention
# print(t.__code)    # ABSTURZ: AttributeError!
print(t._Tresor__code) # Funktioniert technisch (ausgehebeltes Name Mangling)
```

■ Getter und Setter mit @property

Um den direkten Zugriff auf Attribute zu kontrollieren und gleichzeitig eine saubere, intuitive Syntax zu wahren, verwendet man Properties. Sie erlauben es, Methoden wie Attribute anzusprechen (ohne Klammern `()`), wodurch nachgelagerte Datenvalidierungen realisiert werden können.

```
class Konto:
    def __init__(self, inhaber, kontostand):
        self.inhaber = inhaber
        self.__kontostand = kontostand

    @property
    def kontostand(self):
        '''Der Getter: Gibt den privaten Kontostand zurück.'''
        return self.__kontostand

    @kontostand.setter
    def kontostand(self, neuer_wert):
        '''Der Setter: Validiert den neuen Wert vor dem Schreiben.'''
        if neuer_wert < 0:
            raise ValueError("Ein Kontostand kann nicht negativ sein!")
        self.__kontostand = neuer_wert

k = Konto("Bob", 100)
print(k.kontostand)    # Ruft den Getter auf -> 100 (keine Klammern!)
k.kontostand = 250      # Ruft den Setter auf
# k.kontostand = -50    # Wirft ValueError!
```

■ Vererbung, Überschreiben & super()

Vererbung erlaubt es einer Kindklasse (Subklasse), alle Attribute und Methoden einer Elternklasse (Basisklasse) zu übernehmen, zu erweitern oder anzupassen.

Methoden überschreiben: Die Kindklasse definiert eine Methode mit identischem Namen wie die Elternklasse. Beim Aufruf auf einer Instanz der Kindklasse wird die neue Methode ausgeführt.

`super()`: Ermöglicht den expliziten Aufruf von Methoden der Elternklasse aus der Kindklasse heraus. Nahezu obligatorisch in der `__init__`-Methode der Kindklasse, um die Basisinitialisierung sicherzustellen.

```
class Person:
    def __init__(self, name):
        self.name = name
    def info(self):
        return f"Name: {self.name}"
```

```
class Mitarbeiter(Person):
    def __init__(self, name, gehalt):
        super().__init__(name) # Ruft Konstruktor der Basisklasse Person auf
        self.gehalt = gehalt

    def info(self): # Ueberschreiben der Methode
        # Nutzt das Verhalten der Elternklasse und erweitert es
        return f"{super().info()} | Gehalt: {self.gehalt}€"
```

■ Statische Variablen und Methoden

Klassenattribute (Statische Variablen): Werden direkt im Klassenrumpf (außerhalb von Methoden) definiert. Sie existieren nur einmalig pro Klasse und werden von allen Instanzen dieser Klasse geteilt.

@staticmethod: Dekariert eine Methode, die keinen Zugriff auf das Objekt (self) oder die Klasse (cls) benötigt. Verhält sich wie eine reguläre, isolierte Funktion, die lediglich im Namensraum der Klasse verortet ist.

@classmethod: Erhält als impliziten ersten Parameter die Klasse selbst (cls) anstelle der Instanz. Wird primär für alternative Konstruktoren (Factory Methods) verwendet.

```
class Angestellter:
    # Klassenattribut (statische Variable)
    anzahl_angestellte = 0

    def __init__(self, name):
        self.name = name
        Angestellter.anzahl_angestellte += 1

    @staticmethod
    def ist_volljaehrig(alter):
        # Unabh ngig von Instanz- oder Klassendaten
        return alter >= 18

    @classmethod
    def von_string(cls, daten_string):
        # Alternativer Konstruktor. cls repr sentiert die Klasse 'Angestellter'
        name = daten_string.strip()
        return cls(name) # Instanziiert neues Objekt

a1 = Angestellter("Anna")
a2 = Angestellter.von_string("Ben") # Aufruf ueber Classmethod
print(Angestellter.anzahl_angestellte) # -> 2
print(Angestellter.ist_volljaehrig(20)) # -> True
```

■ Abstract Base Classes (ABC)

Abstrakte Basisklassen dienen als strikte strukturelle Vorlage f r Unterklassen. Von einer ABC k nnen keine direkt instanziierten Objekte erzeugt werden. Sie erzwingt, dass Kindklassen bestimmte Methoden zwingend implementieren m ssen.

Umsetzung mittels des integrierten Moduls abc durch Vererbung von ABC und Kennzeichnung der Pflichtmethoden mit dem Dekorator @abstractmethod.

```
from abc import ABC, abstractmethod

class Form(ABC): # Abstrakte Basisklasse
    @abstractmethod
    def berechne_flaeche(self) -> float:
        '''Muss von jeder konkreten Unterklasse implementiert werden!'''
        pass

class Quadrat(Form):
    def __init__(self, seite):
        self.seite = seite
    def berechne_flaeche(self): # Verpflichtung erf llt
        return self.seite ** 2

# f = Form() # FEHLER: TypeError (Can't instantiate abstract class...)
q = Quadrat(5) # Funktioniert einwandfrei
```

Passives Wissen (Konzeptionelles Verständnis)

In diesem Bereich wird primär ein passives Text- und Codeverständnis erwartet (wichtig für Multiple-Choice-Fragen oder Code-Analysen).

■ Generatoren & Dict Comprehension

Generatoren: Funktionen, die das Schlüsselwort `yield` anstelle von `return` nutzen. Sie geben Werte inkrementell per Lazy Evaluation (träge Auswertung) zurück. Sie merken sich ihren internen Ausführungszustand und sparen massiv Arbeitsspeicher, da nicht die gesamte Sequenz auf einmal im Speicher gehalten werden muss.

Dict Comprehension: Analog zu List Comprehensions, erzeugt jedoch Dictionaries. Syntax: `{key_expr: value_expr for item in iterable}`.

```
# Generator-Funktion
def zahlen_generator(maximum):
    n = 0
    while n < maximum:
        yield n
        n += 1

gen = zahlen_generator(100000) # Erzeugt Generator-Objekt, berechnet noch nichts
print(next(gen)) # -> 0 (Berechnet ersten Wert on-demand)

# Dict Comprehension
namen = ["Alice", "Bob"]
laengen_dict = {n: len(n) for n in namen} # {"Alice": 5, "Bob": 3}
```

■ Binärdateien & zugehörige Datentypen

Binärdateien: Werden mit dem Modus-Zusatz `b` geöffnet (z.B. `open("bild.png", "rb")`). Es findet keine implizite En- oder Dekodierung (z.B. UTF-8) statt. Die Rohdaten werden unverändert transferiert.

Datentyp `bytes`: Eine unveränderbare (immutable) Sequenz von Bytes (Ganzzahlen im Bereich 0-255). Literale Deklaration via Präfix `b'...'`.

Datentyp `bytearray`: Die veränderbare (mutable) Variante des `bytes`-Objekts.

```
# Bytes-Objekt
b_daten = b"Hello"
# b_daten[0] = 72 # FEHLER: bytes ist immutable

# bytearray-Objekt
ba_daten = bytearray(b"Hello")
ba_daten[0] = 65 # Ersetzt 'H' durch 'A' -> bytearray(b"Aello")
```

■ Lambda-Funktionen und funktionale Werkzeuge

Lambda-Funktionen: Anonyme (namenlose) Kurz-Funktionen, die aus genau einem einzigen Ausdruck bestehen. Syntax: `lambda param1, param2: ausdruck`.

`map(func, iterable)`: Wendet die Funktion `func` auf jedes Element des Iterables an.

`filter(predicate, iterable)`: Filtert Elemente heraus, für die die Funktion `predicate` nicht `True` zurückgibt.

`reduce(func, iterable)`: Reduziert ein Iterable schrittweise von links nach rechts auf einen einzigen Endwert (erfordert `from functools import reduce`).

```
# Lambda als kompakter Filter/Mapper
zahlen = [1, 2, 3, 4, 5]

# Verdoppeln aller Zahlen via map
verdoppelt = list(map(lambda x: x * 2, zahlen)) # [2, 4, 6, 8, 10]

# Nur ungerade Zahlen via filter
ungerade = list(filter(lambda x: x % 2 != 0, zahlen)) # [1, 3, 5]
```

■ Variable Parameterlisten (*args und kwargs)

Ermöglichen es Funktionen, eine variable Anzahl von Argumenten entgegenzunehmen:

*args (Positional Arguments): Nimmt beliebig viele positionelle Argumente auf und stellt sie innerhalb der Funktion als Tupel zur Verfügung.

kwargs (Keyword Arguments): Nimmt beliebig viele benannte Argumente auf und stellt sie innerhalb der Funktion als Dictionary zur Verfügung.

```
def flexible_funktion(*args, **kwargs):
    print(args)    # Ein Tupel aller positionellen Argumente
    print(kwargs)  # Ein Dictionary aller benannten Argumente

flexible_funktion(1, 2, "Test", status=True, wert=9.5)
# args: (1, 2, "Test")
# kwargs: {"status": True, "wert": 9.5}
```

■ UML als Beschreibungssprache für OOP

In der Unified Modeling Language (UML) werden Klassen als dreigeteilte Rechtecke visualisiert:

Oberer Abschnitt: Der Klassenname.

Mittlerer Abschnitt: Die Attribute der Klasse.

Unterer Abschnitt: Die Methoden der Klasse.

Sichtbarkeitskennzeichnungen im UML-Diagramm:

1. steht für public

steht für protected

• steht für private

Beziehungen:

Vererbung (Is-A-Beziehung): Wird dargestellt durch eine Linie mit einer geschlossenen, leeren Pfeilspitze, die von der Kindklasse zur Basisklasse zeigt.

Assoziation (Has-A-Beziehung): Eine einfache Verbindungslinie zwischen Klassen, die anzeigt, dass Objekte miteinander interagieren oder ein Objekt eine Referenz auf ein anderes hält.